Analysis: eval_gpt41_100rows

Summary

Your evaluation shows a **legally compliant, structurally sound intake agent** with perfect practice area classification (100% accuracy, F1=1.0). However, there's room to improve professional tone (3.54/5) and category-level accuracy (80%).

Key Findings

Strengths:

- **Perfect legal compliance** (5/5) and practice area routing (100% accuracy, recall, F1)
- **Strong response structure** (5/5) and information completeness (4.71/5)
- **No errors** across 100 evaluations
- **Efficient token caching** (up to 2,304 cached tokens/trace) reducing costs
- **Reasonable latency** (mean: 3.29s, p90: 3.9s)
- **Low cost** ($0.00219/intake)

Weaknesses:

- **Professional tone** (3.54/5) — lowest-scoring quality metric
- **Category accuracy** (80%) and macro recall (75.4%) — significant gap vs. practice area metrics
- **No trace-level assessments** — evaluation results not linked to individual traces for debugging
- **No persisted evaluation dataset** — can't re-run or compare future evals
- **No scheduled scorers** — no ongoing quality monitoring

Architecture Insights

Your LangGraph agent executes a multi-turn workflow:

1. **Agent → Tools** (semantic_retrieval, conflict_check)
2. **Agent → Tools** (case_routing)
3. **Final agent response**

Recommended Improvements

1. Improve Professional Tone (Priority: High)

Problem: 3.54/5 suggests inconsistent formality or client-facing language.

Actions:

- **Analyze failing cases**: Search traces with low professional_tone scores to identify patterns
- `# Find traces with assessments, filter by low scores`
- `traces = mlflow.search_traces(`
- `    experiment_ids=["2234221110237340"],`
- `    filter_string="assessments.`professional_tone` < 4"`
- `)`
- **Refine system prompt**: Add explicit tone guidelines (e.g., "Use formal language," "Avoid slang," "Express empathy professionally")
- **Add few-shot examples** of professional vs. unprofessional responses in your prompt

2. Fix Category Classification Gap (Priority: High)

Problem: 80% accuracy vs. 100% for practice areas suggests confusion within subcategories.

Actions:

- **Inspect confusion matrix**: Identify which categories are being misclassified
- `# Retrieve evaluation results and analyze category confusion`
- `%python`
- `%pip install --upgrade mlflow[databricks]`
- `dbutils.library.restartPython()`
- `import mlflow`
-
- `# Get all traces from the evaluation period`
- `traces = mlflow.search_traces(`
- `    experiment_ids=["2234221110237340"],`
- `    filter_string="timestamp_ms >= 1719011328588 AND timestamp_ms <= 1719011469998",`
- `    max_results=100`
- `)`
-
- `# If assessments include category predictions, analyze them`
- `# Otherwise, extract from trace outputs`
- **Enhance case_routing tool**: Provide more granular category definitions and examples
- **Add category-specific validation**: Have the agent verify its category choice before finalizing

3. Link Assessments to Traces (Priority: Medium)

Problem: Run metrics exist, but no trace-level assessments for debugging individual failures.

Actions:

```
# When running mlflow.genai.evaluate(), ensure assessments are written to traces
```

```
result = mlflow.genai.evaluate(
    model=your_model,
    data=eval_data,
    scorers=[
        mlflow.genai.LLMJudgeScorer(name="professional_tone", ...),
        mlflow.genai.LLMJudgeScorer(name="information_completeness", ...),
        # ... other scorers
    ],
    # Assessments will be automatically linked to traces
)
```

After evaluation, you can filter traces by score:

```
# Find low-scoring traces
low_tone_traces = mlflow.search_traces(
    experiment_ids=["2234221110237340"],
    filter_string="assessments.`professional_tone` < 4"
)
```

4. Persist Evaluation Dataset (Priority: Medium)

Problem: No dataset found in the experiment — can't reproduce or compare evaluations.

Actions:

```
# Save your evaluation dataset to MLflow
dataset = mlflow.data.from_pandas(
    df=eval_dataframe,
    source="legal_intake_eval_v1",
    name="legal_intake_100_cases"
)

with mlflow.start_run():
    mlflow.log_input(dataset, context="eval")
```

5. Set Up Continuous Monitoring (Priority: Medium)

Problem: No scheduled scorers — quality regressions won't be caught automatically.

Actions:

```
# Register scorers for ongoing monitoring
scorer = mlflow.genai.LLMJudgeScorer(
    name="professional_tone",
    llm_model="databricks-meta-llama-3.3-70b-instruct",
    # ... scorer config
)

scorer.register(name="professional_tone").start(
    sampling_config=mlflow.genai.SamplingConfig(sample_rate=0.1)  # 10% of
traces
)
```

This will automatically score 10% of production traces for ongoing quality tracking.

6. Optimize Latency (Priority: Low)

Current: p90=3.9s is acceptable for intake, but could be faster.

Actions:

- **Parallelize tool calls**: Run `semantic_retrieval` and `conflict_check` concurrently
- **Reduce prompt size**: Your traces show 3,123 input tokens — consider pruning system prompt
- **Use a faster model** for routing decisions (keep GPT-4 for final response)
- **Monitor cache hit rate**: You're already getting cache benefits (2,304 cached tokens) — ensure prompts are stable to maximize this

7. Save Evaluation Artifacts (Priority: Low)

Problem: No artifacts saved — harder to share/review results.

Actions:

```
# After evaluation, log results
with mlflow.start_run(run_id="your_eval_run_id"):
    # Log confusion matrices, score distributions, etc.
    mlflow.log_table(confusion_matrix_df, "category_confusion_matrix.json")
    mlflow.log_metrics({"category_f1_business_law": 0.85, ...})  # Per-
category metrics
```